

Programming Techniques 2026–2027

Lecture 4: Data Type Precision, Programs, Modules, and Procedures

Hannu Parviainen

Universidad de la Laguna

October 5, 2026

Data Type Precision

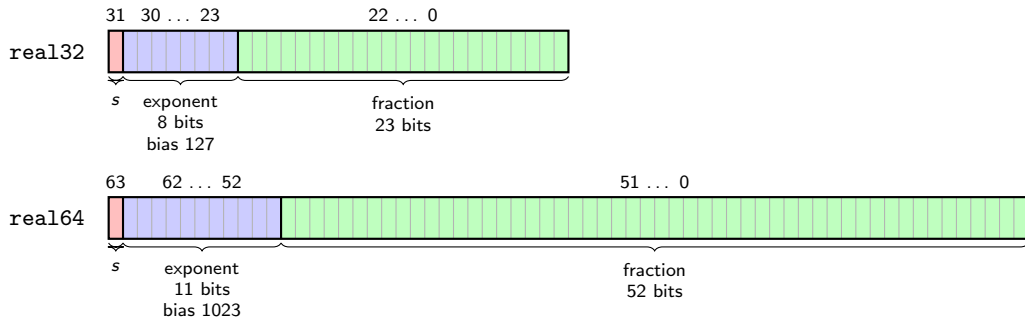
Floating-point numbers

- ▶ Computers store reals in binary scientific notation, like 6.022×10^{23} but in base 2:

$$x = (-1)^s \times 1.f \times 2^{e-\text{bias}}$$

- ▶ A real has a fixed number of bits, split into three fields: the sign s , the exponent e , and the fraction f .
- ▶ The leading 1 is implied and not stored. The bias lets the stored exponent be a non-negative integer.
- ▶ The fixed number of bits sets two limits:
 - ▶ the fraction bits set the **precision**: how many significant digits a number has,
 - ▶ the exponent bits set the **range**: how large and how small a number can be.

Bit layout of 32- and 64-bit reals



- ▶ 32-bit real (*single precision*): about 7 significant digits, range about $10^{\pm 38}$.
- ▶ 64-bit real (*double precision*): about 16 significant digits, range about $10^{\pm 308}$.
- ▶ In Fortran: `real(real32)` and `real(real64)`, explained at the end of this section.

How precision matters: the gaps grow with magnitude

- ▶ Reals are not evenly spaced. With a fixed number of significant digits, the gap to the next real grows with the size of the number (`spacing(x)`).
- ▶ Example: a Julian date, $JD = 2460000.5$. A 32-bit real can only store it in steps of 0.25 days, that is, **6 hours**. A 64-bit real resolves $40\ \mu s$.
- ▶ Fix: keep the numbers small, e.g. store times relative to a reference epoch, $t - t_0$.

```
program jd_precision
  use iso_fortran_env, only: real32, real64
  implicit none
  real(real32) :: jd_s = 2460000.5
  real(real64) :: jd_d = 2460000.5_real64
  print *, spacing(jd_s)           ! 0.250000000 days
  print *, spacing(jd_d) * 86400 ! 4.02E-05 seconds
end program jd_precision
```

How precision matters: rounding errors add up

- ▶ Every operation rounds its result, and the rounding errors add up.
- ▶ Adding 0.1 ten million times should give 10^6 . With 32-bit reals, the result is off by 9%.
- ▶ The same happens with $t = t + dt$ in a time integration.
- ▶ Fix: compute $t = i*dt$ instead, or use 64-bit reals.

```
program accumulate
  use iso_fortran_env, only: real32, real64
  implicit none
  integer :: i
  real(real32) :: s32 = 0.0
  real(real64) :: s64 = 0.0_real64
  do i = 1, 10000000
    s32 = s32 + 0.1_real32
    s64 = s64 + 0.1_real64
  end do
  print *, s32      ! 1087937.00
  print *, s64      ! 999999.99983897537
end program accumulate
```

How precision matters: subtraction cancels digits

- ▶ Subtracting two nearly equal numbers cancels their leading digits. What is left is mostly rounding error.
- ▶ Example: for $x = 10^{-4}$, $\cos x$ rounds to exactly 1.0 in 32-bit, so $1 - \cos x$ gives 0 instead of 5×10^{-9} .
- ▶ Fix: rewrite the formula. The equivalent $2 \sin^2(x/2)$ has no subtraction and is accurate even in 32-bit.

```
program cancellation
  implicit none
  real :: x = 1.0e-4
  print *, 1.0 - cos(x)           ! 0.00000000
  print *, 2.0 * sin(x/2)**2     ! 4.99999953E-09
end program cancellation
```

How precision matters: the range is limited

- ▶ Results too large for the type become Infinity (*overflow*). The solar mass squared in kg^2 , about 4×10^{60} , overflows a 32-bit real.
- ▶ Results too close to zero become 0 (*underflow*).
- ▶ Integers overflow without any warning: 50000^2 wraps around to a negative number in a 32-bit integer.
- ▶ Fix: use 64-bit types, or units that keep numbers moderate ($M_\odot$, au, days).

```
program range_limits
  use iso_fortran_env, only: real32, int32
  implicit none
  real(real32) :: m_sun = 1.989e30 ! kg
  real(real32) :: x = -104.0
  integer(int32) :: n = 50000
  print *, m_sun**2 ! Infinity
  print *, exp(x) ! 0.00000000
  print *, n*n ! -1794967296
end program range_limits
```

Choosing the precision

Use 64-bit reals by default

About 16 significant digits and a range of $10^{\pm 308}$ are enough for most scientific computing, at a small cost.

32-bit reals have their place

They halve the memory use and are faster, especially on GPUs. Use them for large data sets when 7 digits are enough.

More bits do not fix bad numerics

Keep numbers moderate (scaled units, $t - t_0$), rewrite formulas that cancel, and never compare reals with `==`.

Kind parameters

- ▶ In Fortran, the precision of a type is set by its **kind** parameter: `integer(kind=k)`, `real(kind=k)`, or simply `real(k)`.
- ▶ The kind is an integer, but the standard does not say what the numbers mean. Each compiler chooses its own:
 - ▶ GNU Fortran: the kind is the number of bytes, so `real(8)` is a 64-bit real.
 - ▶ NAG Fortran (by default): kinds 1 and 2 are the 32- and 64-bit reals, and `real(8)` does not exist.
- ▶ Hard-coded kind numbers are therefore not portable. The next slides show three portable alternatives.
- ▶ C has a similar problem: the size of `int` and `long` depends on the platform.

Kind numbers in GNU Fortran

- ▶ The kind is the number of bytes:
 - ▶ integer and logical: 1, 2, 4, 8, 16
 - ▶ real and complex: 4, 8, 10, 16
- ▶ A complex number holds two reals, so `complex(4)` takes 8 bytes.
- ▶ The default kind is 4 for integer, logical, real, and complex, and 1 for character. `double precision` is `real(8)`.
- ▶ Fine for quick tests, but prefer the portable methods that follow.

```
real(kind=4)      :: a  ! 32-bit real
real(kind=8)      :: b  ! 64-bit real
integer(kind=4)   :: i  ! 32-bit integer
integer(kind=8)   :: j  ! 64-bit integer
```

Portable kinds by precision and range

- ▶ State what you need, and let the compiler pick the kind:
 - ▶ `selected_int_kind(r)`: the smallest integer kind that holds all n with $-10^r < n < 10^r$.
 - ▶ `selected_real_kind(p, r)`: a real kind with at least p significant digits and an exponent range of at least $10^{\pm r}$.
- ▶ `selected_real_kind(15, 307)` gives a 64-bit real with any common compiler.
- ▶ If no kind fits, the result is negative and the declaration fails to compile.

```
program ex3a
  implicit none
  integer, parameter :: ik = selected_int_kind(15)
  integer, parameter :: dp = selected_real_kind(15, 307)
  integer(ik) :: n
  real(dp) :: x
  print *, huge(n)                ! 9223372036854775807
  print *, precision(x), range(x) ! 15 307
end program ex3a
```

Portable kinds by size: ISO_FORTRAN_ENV

- ▶ The intrinsic module `iso_fortran_env` names the kinds by their size in bits: `int8`, ..., `int64`, `real32`, `real64`, `real128`.
- ▶ The module is from Fortran 2003, and these names from Fortran 2008.
- ▶ They fix the number of bits, not the precision, but in practice they are the 32- and 64-bit reals shown earlier.
- ▶ Common practice: define the kind once as a named constant (`dp` in the example) and use it everywhere. Later in this lecture, we put it in a module.

```
program ex3b
  use iso_fortran_env, only: int32, int64, real32, real64
  implicit none
  integer, parameter :: dp = real64
  integer(int32) :: i
  integer(int64) :: j
  real(real32) :: x
  real(dp) :: y
  print *, huge(i), huge(j)
  print *, tiny(x), huge(x)
  print *, tiny(y), huge(y)
end program ex3b
```

Trap: literals have a kind too

- ▶ A real literal without a kind, such as 0.1, is a default real, that is, 32-bit.
- ▶ It is rounded to 32 bits before it is converted to 64 bits, so it keeps the 32-bit rounding error.
- ▶ Always give real literals a kind: 0.1_dp, 6.674e-11_dp. This includes literals in expressions, such as $x * 0.1$.
- ▶ Older code writes 0.1d0, where the exponent letter d means double precision.

```
program literal_kinds
  use iso_fortran_env, only: real64
  implicit none
  integer, parameter :: dp = real64
  real(dp), parameter :: pi = 3.141592653589793_dp
  real(dp) :: a = 0.1      ! 32-bit literal
  real(dp) :: b = 0.1_dp   ! 64-bit literal
  print *, a      ! 0.10000000149011612
  print *, b      ! 0.10000000000000001
  print *, pi
end program literal_kinds
```

Kinds for C interoperability: ISO_C_BINDING

- ▶ The intrinsic module `iso_c_binding` (Fortran 2003) provides kinds that match the C types: `c_int`, `c_long`, `c_float`, `c_double`, ...
- ▶ Use them when Fortran and C code exchange data, e.g. when calling a C library.
- ▶ On common platforms, `c_float` and `c_double` are the same as `real32` and `real64`.

```
program ex3c
  use iso_c_binding, only: c_float, c_double
  implicit none
  real(c_float) :: x
  real(c_double) :: y
  print *, tiny(x), huge(x)
  print *, tiny(y), huge(y)
end program ex3c
```

Program Units

Fortran program units

Program units

- ▶ **program**: where execution begins. Every executable has exactly one.
- ▶ **module**: a container for constants, variables, types, and procedures. Other program units get access to it with `use`.

Procedures

- ▶ **subroutine**: performs a task. It is invoked with `call` and returns no value.
- ▶ **function**: returns a value, so it is used inside an expression.

Where procedures live

Procedures are placed inside a program or a module, after a `contains` statement.

The main program

- ▶ Execution starts at the first executable statement of the program.
- ▶ The order is fixed: use statements, implicit none, declarations, and then executable statements.
- ▶ An optional contains section holds *internal procedures*, which only this program can call.
- ▶ Here negative is an internal function. We return to functions later in this lecture.

```
program main
  implicit none
  real :: x                ! declarations
  read *, x                ! executable statements
  print *, floor(x)        ! intrinsic function
  print *, negative(x)     ! internal function
contains
  real function negative(a)
    real, intent(in) :: a
    negative = -a
  end function negative
end program main
```

```
$ echo 2.7 | ./a.out
      2
 -2.70000005
```

Modules

A first module: kinds

- ▶ A module is a named container. The simplest ones hold only constants.
- ▶ A use statement makes the contents of a module available. It goes before `implicit none`.
- ▶ This `kinds` module does what the precision section recommended: it defines the kind once, in one place.
- ▶ Every part of the program then uses `real(dp)`, and changing the precision of the whole program means changing one line.

```
module kinds
  use iso_fortran_env, only: real32, real64
  implicit none
  integer, parameter :: sp = real32
  integer, parameter :: dp = real64
end module kinds

program use_kinds
  use kinds, only: dp
  implicit none
  real(dp) :: x = 0.1_dp
  print *, x      ! 0.10000000000000001
end program use_kinds
```

Modules with procedures

- ▶ Most modules are libraries: they collect related constants, types, functions, and subroutines.
- ▶ The procedures go after contains.
- ▶ A module can share a file with the program, as here, but it must come first. Usually each module has its own file.
- ▶ Modules are compiled separately, so many programs can use the same one.

```
module mymath
  implicit none
  real, parameter :: pi = 3.14159
contains
  real function square(a)
    real, intent(in) :: a
    square = a**2
  end function square
end module mymath

program ex1
  use mymath
  implicit none
  print *, square(2.3), pi
end program ex1
```

Compiling a program that uses a module

- ▶ With the module and the program in `ex1.f90`, compiling produces the executable `ex1` and a module file, `mymath.mod`.
- ▶ The `.mod` file describes the contents of the module, much like a header file in C. The compiler reads it whenever it meets use `mymath`.

```
$ gfortran -o ex1 ex1.f90
$ ls
ex1  ex1.f90  mymath.mod
$ ./ex1
      5.28999996      3.14159012
```

Modules in separate files

- ▶ Now `mymath.f90` holds the module and `ex1.f90` the program.
- ▶ The module must be compiled first, because the program needs its `.mod` file.
- ▶ Either compile it separately and link the object file, or give both files in the right order.
- ▶ For larger projects, use a **Makefile** (covered later in the course).

```
$ gfortran -c mymath.f90
$ gfortran -o ex1 ex1.f90 mymath.o

$ gfortran -o ex1 mymath.f90 ex1.f90
```

Listing 1: mymath.f90

```
module mymath
  implicit none
  real, parameter :: pi = 3.14159
contains
  real function square(a)
    real, intent(in) :: a
    square = a**2
  end function square
end module mymath
```

Listing 2: ex1.f90

```
program ex1
  use mymath
  implicit none
  print *, square(2.3), pi
end program ex1
```

Controlling access: private, public, and only

- ▶ Everything in a module is public by default.
- ▶ A bare private statement hides everything from the users of the module. A public :: statement then lists what the module offers.
- ▶ On the other side, use shapes, only: sphere_volume imports only the listed names.
- ▶ This avoids name clashes and shows where each name comes from. We have already used it with iso_fortran_env.

```
module shapes
  implicit none
  private
  public :: sphere_volume
  real, parameter :: pi = 3.14159    ! private
contains
  real function sphere_volume(r)
    real, intent(in) :: r
    sphere_volume = 4.0 / 3.0 * pi * r**3
  end function sphere_volume
end module shapes

program ex2
  use shapes, only: sphere_volume
  implicit none
  print *, sphere_volume(2.0)    ! 33.5102959
end program ex2
```

Procedures

Subroutines

- ▶ A subroutine performs a task and is invoked with `call`.
- ▶ It returns no value. Any results go back through its arguments.
- ▶ The general form:

```
subroutine name(arguments)
  ! declarations of the arguments
  ! local declarations
  ! executable statements
end subroutine name
```

```
program figures
  implicit none
  call output_figures([1.0, 2.0])
contains
  subroutine output_figures(numbers)
    real, intent(in) :: numbers(:)
    print *, 'Here are the figures', numbers
  end subroutine output_figures
end program figures
```

```
$ ./a.out
Here are the figures      1.00000000      2.00000000
```

Functions

- ▶ A function returns a value, so it is used inside an expression.
- ▶ There are two ways to name the returned value:
 - ▶ with `result(res)`, where `res` is declared like any other variable,
 - ▶ with the name of the function, whose type is then given in front of function.
- ▶ Both do the same thing. We use the `result` form for the recursive functions below.

```
program functions
  implicit none
  ! Both functions return 5.0
  print *, length_a(3.0, 4.0), length_b(3.0, 4.0)
contains
  function length_a(a, b) result(res)
    real, intent(in) :: a, b
    real :: res
    res = sqrt(a**2 + b**2)
  end function length_a

  real function length_b(a, b)
    real, intent(in) :: a, b
    length_b = sqrt(a**2 + b**2)
  end function length_b
end program functions
```

Arguments and intent

- ▶ The names in the definition of a procedure are its *dummy arguments*. The call supplies the *actual arguments*.
- ▶ Declare the intent of every dummy argument:
 - ▶ `intent(in)`: only read,
 - ▶ `intent(out)`: set by the procedure,
 - ▶ `intent(inout)`: read and modified.
- ▶ The compiler then catches mistakes, such as assigning to an `intent(in)` argument.
- ▶ `v(:)` is an *assumed-shape* array: it takes its shape from the caller.

```
module vectors
  implicit none
contains
  subroutine scale(alpha, v)
    real, intent(in)    :: alpha
    real, intent(inout) :: v(:)
    v = alpha * v
  end subroutine scale
end module vectors

program scale_vector
  use vectors, only: scale
  implicit none
  real :: v(3) = [1.0, 2.0, 3.0]
  call scale(2.0, v)
  print *, v      ! 2.0  4.0  6.0
end program scale_vector
```

Recursive functions

- ▶ A recursive function calls itself.
- ▶ This suits problems that are defined in terms of smaller versions of themselves, such as the factorial, or walking through a tree.
- ▶ It must be declared recursive. Fortran 2018 makes this the default, but gfortran still requires the keyword.
- ▶ It needs a case that ends the recursion. Otherwise it never stops.

```
program recursion
  implicit none
  print *, factorial(5), factorial(10)    ! 120  3628800
contains
  recursive function factorial(n) result(f)
    integer, intent(in) :: n
    integer :: f
    if (n <= 1) then
      f = 1
    else
      f = n * factorial(n - 1)
    end if
  end function factorial
end program recursion
```

Pure functions

- ▶ A pure function has no side effects. It cannot change its arguments, which must all be `intent(in)`, change module variables, or do I/O.
- ▶ The same arguments therefore always give the same result.
- ▶ The compiler checks this promise, and can then run calls in any order. This makes pure functions safe to use in parallel code.

```
program purity
  implicit none
  print *, length(3.0, 4.0)    ! 5.0
contains
  pure function length(x, y) result(r)
    real, intent(in) :: x, y
    real :: r
    r = sqrt(x**2 + y**2)
  end function length
end program purity
```

Elemental functions

- ▶ An elemental function is written for scalars, but can also be applied to arrays, element by element.
- ▶ This is how intrinsic functions such as `sin` and `exp` work on arrays.
- ▶ The actual arguments may be scalars or conformable arrays.
- ▶ `elemental` implies `pure`.

```
module squares
  implicit none
contains
  elemental real function square(x)
    real, intent(in) :: x
    square = x * x
  end function square
end module squares

program elemental_example
  use squares, only: square
  implicit none
  real :: x(3) = [1.0, 2.0, 3.0]
  print *, square(2.0)    ! 4.0
  print *, square(x)      ! 1.0  4.0  9.0
end program elemental_example
```

Exercises: machine epsilon

Tasks

1. Machine epsilon is the gap between 1.0 and the next larger real. Find it for 32-bit reals with a loop: start from $\text{eps} = 1.0$ and keep halving it for as long as $1.0 + \text{eps}/2 > 1.0$.
2. Do the same for 64-bit reals.
3. Compare your two results with the intrinsic function `epsilon(x)`.
4. How many significant decimal digits does each result correspond to?

Exercises: the order of a sum

Tasks

1. Compute the sum $\sum_{k=1}^N 1/k$ for $N = 10^7$ with 32-bit reals, adding the terms from $k = 1$ upwards.
2. Compute the same sum again, now adding the terms from $k = N$ downwards.
3. Compute it once more with 64-bit reals, and use that as the reference. Which 32-bit result is closer?
4. Explain the difference. Hint: think about the size of the gap between neighbouring reals when a small term is added to a large sum.

Exercises: a quadratic equation

Tasks

1. Write a program that solves $ax^2 + bx + c = 0$ with the standard formula

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

using 32-bit reals.

2. Test it with $a = 1$, $b = -10^4$, and $c = 1$. The exact roots are close to 10^4 and 10^{-4} . What do you get?
3. Explain what went wrong with the smaller root.
4. Fix it without changing the precision. Hint: the product of the two roots is c/a .
5. Repeat with 64-bit reals. How many digits of the smaller root does the standard formula get right now?

Exercises: modules, subroutines, and functions

Tasks

1. Write a module `mymath` in a file `mymath.f90` that contains:
 - ▶ a parameter `pi = 3.14159`,
 - ▶ a function `circle_area(r)` that returns the area of a circle.
2. Write a main program in a file `test_mymath.f90` that uses the module and prints the area of a circle with radius 2.0.
3. Compile and link the module and the program.
4. Add a `kinds` module that defines `dp`, and change `mymath` and the program to use `real(dp)`. How many digits of the result can you trust now?

Exercises: a subroutine with several results

Tasks

1. Write a module `stats` with a subroutine `mean_and_std(x, mean, std)` that returns the mean and the sample standard deviation of a real array `x` of any size.
2. Give every dummy argument the right `intent`, and make the array `assumed-shape`.
3. Write a main program that calls the subroutine with the magnitudes `[4.2, 3.9, 4.5, 4.1, 4.3]` and prints the results.
4. Add a statement to the subroutine that changes `x`. What does the compiler say?
5. Make the module `private`, export only the subroutine, and import it with `only`.

Exercises: an elemental function

Tasks

1. Write a module `photometry` with an elemental function `flux_to_mag(flux)` that returns the magnitude $m = -2.5 \log_{10} F$.
2. Call it from a main program with a single flux, and then with an array of fluxes.
3. Add a `print` statement inside the function. What does the compiler say, and why?
4. What happens if one of the fluxes is zero or negative? Protect the calculation in the main program with a `where` construct.

Exercises: recursive functions

Tasks

1. Write a recursive function `factorial(n)` that computes $n!$.
2. Test the function by computing $5!$ and $10!$.
3. Modify the function to handle invalid inputs (e.g. $n < 0$).
4. Write a recursive function `fibonacci(n)` that returns the n th Fibonacci number.
5. Compare the recursive implementation with an iterative one.